word2vec 핵심 알고리즘 구현 분석 (Core Algorithm Implementation Analysis)

📚 문서 네비게이션 (Document Navigation)

| 이전 문서 | 현재 문서 | 다음 문서 |

|----------|----------|----------|

| ← 2. 구현체 구조 분석 | **3. 핵심 알고리즘 분석** | 4. 활용 사례 및 확장 → |

관련 문서:

- 구현체 구조 분석 - Python-C 인터페이스 구조
- 활용 사례 및 확장 - 실제 활용 방법

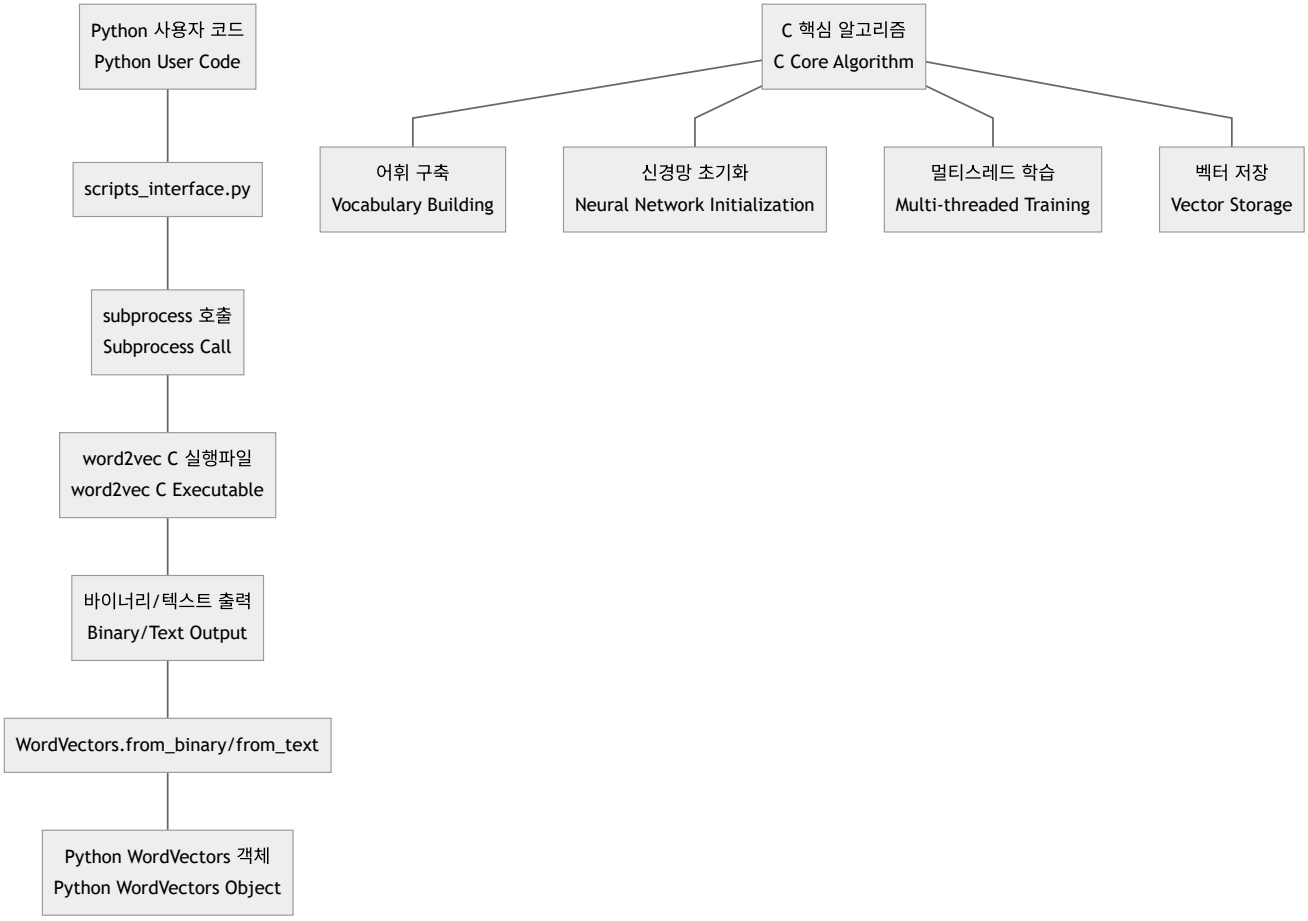
개요 (Overview)

이 문서는 word2vec의 핵심 알고리즘이 C 코드에서 어떻게 구현되고, Python 인터페이스를 통해 어떻게 연결되는지를 상세히 분석합니다. word2vec은 단어를 고차원 벡터 공간에 매핑하는 신경망 기반 알고리즘으로, 효율적인 학습을 위해 C로 구현된 핵심 부분과 사용자 편의성을 위한 Python 래퍼로 구성되어 있습니다.

한국어 비유: 마치 고성능 스포츠카의 엔진(C 알고리즘)과 운전자를 위한 계기판(Python 인터페이스)의 관계와 같습니다. 복잡한 내부 동작은 숨기고, 필요한 정보만 직관적으로 제공합니다.

1. C 코어와 Python 인터페이스 연결 구조 (C Core and Python Interface Connection Structure)

1.1 아키텍처 개요 (Architecture Overview)



1.2 Python-C 인터페이스 구현 (Python-C Interface Implementation)

Python에서 C 코어로의 연결은 `scripts_interface.py` 를 통해 이루어집니다:

```
def  word2vec(train, output, size=100, window=5, sample="1e-3",
hs=0, negative=5, threads=12, iter_=5, min_count=5,
```

```
alpha=0.025, debug=2, binary=False, cbow=1,
save_vocab=None, read_vocab=None, verbose=False):
"""
word2vec 실행 함수 (word2vec execution function)
- C 실행파일을 subprocess로 호출 (Call C executable via subprocess)
- 매개변수를 명령행 인수로 변환 (Convert parameters to command line arguments)
- 학습 완료 후 결과 파일 생성 (Generate result file after training completion)
"""
command = ["word2vec"]
# 매개변수를 명령행 인수로 변환 (Convert parameters to command line arguments)
args = ["-train", "-output", "-size", "-window", ...]
values = [train, output, size, window, ...]
for arg, value in  zip(args, values):
command.append(arg)
command.append(str(value))
# subprocess를 통한 C 프로그램 실행 (Execute C program via subprocess)
run_cmd(command, verbose=verbose)
```

핵심 특징 (Key Features):

- **프로세스 분리 (Process Separation)**: Python과 C 코드가 별도 프로세스로 실행
- **파일 기반 통신 (File-based Communication)**: 결과를 파일로 저장하여 데이터 교환
- **매개변수 매핑 (Parameter Mapping)**: Python 함수 인수를 C 프로그램 명령행 옵션으로 변환

한국어 직관적 설명: Python은 주문서를 작성하고, C 프로그램은 주방에서 요리를 만들어 완성된 음식(벡터 파일)을 내놓는 것과 같습니다.

1.3 데이터 흐름 분석 (Data Flow Analysis)

```
[텍스트 입력] → [C 어휘 구축] → [신경망 학습] → [벡터 저장] → [Python 로딩]

↓ ↓ ↓ ↓ ↓

train.txt vocab_hash[] syn0[], syn1[] output.bin WordVectors
```

각 단계별 설명:

- **텍스트 입력**: 원시 텍스트 데이터 (예: "고양이는 귀여운 동물이다")
- **어휘 구축**: 단어 목록과 빈도 계산 (예: {"고양이": 100, "귀여운": 50, ...})
- **신경망 학습**: 벡터 표현 학습 (예: "고양이" = [0.2, -0.1, 0.8, ...])
- **벡터 저장**: 바이너리 파일로 저장 (빠른 로딩을 위해)
- **Python 로딩**: NumPy 배열로 변환하여 사용

2. 핵심 학습 알고리즘 구현 분석 (Core Learning Algorithm Implementation Analysis)

2.1 어휘 구축 (Vocabulary Building)

C 코드에서 어휘 구축은 다음 단계로 진행됩니다:

```c
// 어휘 구조체 정의 (Vocabulary structure definition)
struct vocab_word {
long  long cn; // 단어 빈도수 (Word frequency)
int *point; // 계층적 소프트맥스용 경로 (Path for hierarchical softmax)
```

```
char *word; // 단어 문자열 (Word string)
char *code; // 허프만 코드 (Huffman code)
char codelen; // 코드 길이 (Code length)
};

// 전역 변수들 (Global variables)
struct vocab_word *vocab; // 어휘 배열 (Vocabulary array)
int *vocab_hash; // 해시 테이블 (Hash table)
const  int vocab_hash_size = 30000000; // 해시 테이블 크기 (Hash table size)
```

어휘 구축 과정 (Vocabulary Building Process):

1. **단어 읽기 및 해싱 (Word Reading and Hashing):**

```
void  ReadWord(char *word, FILE *fin) {
int a = 0, ch;
while (!feof(fin)) {
ch = fgetc(fin);
if ((ch == ' ') || (ch == '\t') || (ch == '\n')) {
if (a > 0) break;
if (ch == '\n') {
strcpy(word, (char *)"</s>"); // 문장 끝 토큰 (End of sentence token)
return;
}
```

```
}
word[a] = ch;
a++;
if (a >= MAX_STRING - 1) a--; // 긴 단어 자르기 (Truncate long words)
}
word[a] = 0;
}


int GetWordHash(char *word) {
unsigned long long hash = 0;
for (int a = 0; a < strlen(word); a++)
hash = hash * 257 + word[a]; // 간단한 해시 함수 (Simple hash function)
return hash % vocab_hash_size;
}
```

한국어 설명:

- `ReadWord()` : 텍스트에서 단어를 하나씩 읽어오는 함수
- `</s>` : 문장의 끝을 나타내는 특수 토큰 (마침표 역할)
- `GetWordHash()` : 단어를 숫자로 변환하여 빠른 검색이 가능하게 하는 함수

2. **어휘 추가 및 빈도 계산 (Vocabulary Addition and Frequency Counting):**

```
int AddWordToVocab(char *word) {
unsigned int hash, length = strlen(word) + 1;
vocab[vocab_size].word = (char *)calloc(length, sizeof(char)); // 메모리 할당
```

```
strcpy(vocab[vocab_size].word, word); // 단어 복사

vocab[vocab_size].cn = 0; // 빈도수 초기화 (Initialize frequency)

vocab_size++;

// 해시 테이블에 등록 (Register in hash table)

hash = GetWordHash(word);

while (vocab_hash[hash] != -1)

hash = (hash + 1) % vocab_hash_size; // 선형 탐사 (Linear probing)

vocab_hash[hash] = vocab_size - 1;

return vocab_size - 1;

}
```

한국어 비유: 마치 사전을 만드는 과정과 같습니다. 새로운 단어를 발견할 때마다 사전에 추가하고, 그 단어가 몇 번 나왔는지 세어둡니다.

3. **빈도순 정렬 및 필터링 (Frequency-based Sorting and Filtering):**

```
void  SortVocab() {

// 빈도수 기준 내림차순 정렬 (</s> 제외) (Sort by frequency in descending order, exc

qsort(&vocab[1], vocab_size - 1, sizeof(struct vocab_word), VocabCompare);

// 최소 빈도 미만 단어 제거 (Remove words below minimum frequency)

for (int a = 0; a < vocab_size; a++) {

if ((vocab[a].cn < min_count) && (a != 0)) {

vocab_size--;

free(vocab[a].word); // 메모리 해제 (Free memory)

} else {
```

```
train_words += vocab[a].cn; // 총 단어 수 계산 (Calculate total word count)

}

}

}
```

한국어 설명: 자주 사용되지 않는 단어들은 제거하여 학습 효율성을 높입니다. 마치 교과서에서 너무 어려운 단어는 빼고 핵심 단어만 남기는 것과 같습니다.#

2.2 신경망 초기화 (Neural Network Initialization)

```
void  InitNet() {

// 입력층 가중치 (syn0) 초기화 (Initialize input layer weights)

posix_memalign((void **)&syn0, 128, vocab_size * layer1_size * sizeof(real));

// 계층적 소프트맥스용 가중치 (syn1) (Weights for hierarchical softmax)

if (hs) {

posix_memalign((void **)&syn1, 128, vocab_size * layer1_size * sizeof(real));

for (int a = 0; a < vocab_size; a++)

for (int b = 0; b < layer1_size; b++)

syn1[a * layer1_size + b] = 0; // 0으로 초기화

}

// 네거티브 샘플링용 가중치 (syn1neg) (Weights for negative sampling)

if (negative > 0) {

posix_memalign((void **)&syn1neg, 128, vocab_size * layer1_size * sizeof(real));
```

```
for (int a = 0; a < vocab_size; a++)
for (int b = 0; b < layer1_size; b++)
syn1neg[a * layer1_size + b] = 0; // 0으로 초기화
}
// 입력층 가중치 랜덤 초기화 (Random initialization of input layer weights)
unsigned long long next_random = 1;
for (int a = 0; a < vocab_size; a++)
for (int b = 0; b < layer1_size; b++) {
next_random = next_random * 25214903917ULL + 11; // 선형 합동 생성기
syn0[a * layer1_size + b] =
(((next_random & 0xFFFF) / 65536.0) - 0.5) / layer1_size;
}
}
```

메모리 정렬 최적화 (Memory Alignment Optimization):

- `posix_memalign` 을 사용하여 128바이트 경계에 메모리 정렬
- SIMD 연산 최적화를 위한 메모리 레이아웃
- 캐시 효율성 향상

한국어 비유:

- `syn0` : 각 단어의 "성격표" (입력 벡터)
- `syn1` : 계층적 소프트맥스용 "판별기"
- `syn1neg` : 네거티브 샘플링용 "판별기"
- 랜덤 초기화: 처음에는 모든 단어가 무작위 성격을 가지고 시작

2.3 멀티스레드 학습 구현 (Multi-threaded Training Implementation)

```
void *TrainModelThread(void *id) {
long  long word_count = 0, sentence_length = 0;
long  long  sen[MAX_SENTENCE_LENGTH + 1]; // 문장 버퍼 (Sentence buffer)
unsigned  long  long next_random = (long  long)id; // 스레드별 시드 (Thread-specif
real *neu1 = (real *)calloc(layer1_size, sizeof(real)); // 은닉층 (Hidden layer)
real *neu1e = (real *)calloc(layer1_size, sizeof(real)); // 오차 벡터 (Error vect
// 파일을 스레드별로 분할하여 읽기 (Divide file among threads)
FILE *fi = fopen(train_file, "rb");
fseek(fi, file_size / num_threads * (long  long)id, SEEK_SET);
while (1) {
// 학습률 동적 조정 (Dynamic learning rate adjustment)
if (word_count - last_word_count > 10000) {
alpha = starting_alpha * (1 - word_count_actual / (real)(iter * train_words + 1)
if (alpha < starting_alpha * 0.0001)
alpha = starting_alpha * 0.0001; // 최소 학습률 보장
}
// 문장 단위 처리 (Process by sentence)
// ... 학습 로직 (Training logic)
}
}
```

한국어 설명:

- 여러 스레드가 동시에 다른 부분의 텍스트를 학습
- 각 스레드는 독립적인 메모리 공간을 가짐
- 학습률이 시간에 따라 점진적으로 감소 (처음에는 크게 변화, 나중에는 미세 조정)

3. Skip-gram과 CBOW 알고리즘 구현 (Skip-gram and CBOW Algorithm Implementation)

3.1 CBOW (Continuous Bag of Words) 구현

CBOW는 주변 단어들로부터 중심 단어를 예측하는 모델입니다:

```
if (cbow) { // CBOW 아키텍처 학습 (CBOW architecture training)

// 입력층 → 은닉층: 컨텍스트 단어들의 평균 (Input → Hidden: Average of context words)

int cw = 0; // 컨텍스트 단어 개수 (Context word count)

for (int a = b; a < window * 2 + 1 - b; a++) {

if (a != window) { // 중심 단어 제외 (Exclude center word)

int c = sentence_position - window + a;

if (c < 0 || c >= sentence_length) continue;

int last_word = sen[c];

if (last_word == -1) continue;

// 컨텍스트 단어 벡터들을 누적 (Accumulate context word vectors)

for (int c = 0; c < layer1_size; c++)

neu1[c] += syn0[c + last_word * layer1_size];

cw++;
```

```
}

}

if (cw) {

// 컨텍스트 벡터들의 평균 계산 (Calculate average of context vectors)

for (int c = 0; c < layer1_size; c++)

neu1[c] /= cw;

// 계층적 소프트맥스 또는 네거티브 샘플링으로 학습

// (Train with hierarchical softmax or negative sampling)

// ...

}

}
```

CBOW의 핵심 특징 (Key Features of CBOW):

- **컨텍스트 평균화 (Context Averaging)**: 주변 단어 벡터들의 평균을 입력으로 사용
- **빠른 학습 (Fast Training)**: 한 번의 forward pass로 여러 단어 처리
- **빈번한 단어에 유리 (Good for Frequent Words)**: 자주 등장하는 단어의 표현 학습에 효과적

한국어 직관적 설명: 마치 여러 친구들의 의견을 종합해서 답을 맞추는 것과 같습니다. "고양이는 ___ 귀엽다"에서 "고양이는"과 "귀엽다"를 보고 빈칸에 들어갈 단어를 추측합니다.

3.2 Skip-gram 구현

Skip-gram은 중심 단어로부터 주변 단어들을 예측하는 모델입니다:

```
else { // Skip-gram 학습 (Skip-gram training)

// 각 컨텍스트 단어에 대해 개별적으로 학습 (Train individually for each context word)

for (int a = b; a < window * 2 + 1 - b; a++) {
```

```
if (a != window) {
int c = sentence_position - window + a;
if (c < 0 || c >= sentence_length) continue;
int last_word = sen[c];
if (last_word == -1) continue;
int l1 = last_word * layer1_size; // 입력 단어 벡터 위치 (Input word vector positi
for (int c = 0; c < layer1_size; c++) neu1e[c] = 0;
// 계층적 소프트맥스 학습 (Hierarchical softmax training)
if (hs) {
for (int d = 0; d < vocab[word].codelen; d++) {
real f = 0;
int l2 = vocab[word].point[d] * layer1_size;
// 은닉층 → 출력층 전파 (Hidden → Output propagation)
for (int c = 0; c < layer1_size; c++)
f += syn0[c + l1] * syn1[c + l2];
// 시그모이드 함수 적용 (Apply sigmoid function)
if (f <= -MAX_EXP) continue;
else  if (f >= MAX_EXP) continue;
else f = expTable[(int)((f + MAX_EXP) * (EXP_TABLE_SIZE / MAX_EXP / 2))];
// 그래디언트 계산 (Calculate gradient)
real g = (1 - vocab[word].code[d] - f) * alpha;
// 역전파: 출력층 → 은닉층 (Backpropagation: Output → Hidden)
for (int c = 0; c < layer1_size; c++)
```

```
neu1e[c] += g * syn1[c + l2];

// 가중치 업데이트: 은닉층 → 출력층 (Weight update: Hidden → Output)

for (int c = 0; c < layer1_size; c++)

syn1[c + l2] += g * syn0[c + l1];

}

}

// 가중치 업데이트: 입력층 → 은닉층 (Weight update: Input → Hidden)

for (int c = 0; c < layer1_size; c++)

syn0[c + l1] += neu1e[c];

}

}

}
```

Skip-gram의 핵심 특징 (Key Features of Skip-gram):

- **개별 예측 (Individual Prediction)**: 각 컨텍스트 단어를 개별적으로 예측
- **희귀 단어에 유리 (Good for Rare Words)**: 적게 등장하는 단어도 효과적으로 학습
- **더 많은 학습 데이터 (More Training Data)**: 하나의 중심 단어로 여러 예측 수행

한국어 직관적 설명: 한 단어를 보고 그 주변에 올 수 있는 모든 단어들을 예측하는 것입니다. "정말"이라는 단어를 보고 “고양이는”, “매우”, “귀엽다” 등이 함께 나올 수 있다고 학습합니다.

4. 계층적 소프트맥스 (Hierarchical Softmax) 구현

4.1 허프만 트리 구축 (Huffman Tree Construction)

계층적 소프트맥스는 어휘 크기에 비례하는 계산 복잡도를 로그 스케일로 줄입니다:

```
void  CreateBinaryTree() {
long  long *count = (long  long *)calloc(vocab_size * 2 + 1, sizeof(long  long))
long  long *binary = (long  long *)calloc(vocab_size * 2 + 1, sizeof(long  long)
long  long *parent_node = (long  long *)calloc(vocab_size * 2 + 1, sizeof(long
// 단어 빈도를 초기값으로 설정 (Set word frequencies as initial values)
for (int a = 0; a < vocab_size; a++) count[a] = vocab[a].cn;
for (int a = vocab_size; a < vocab_size * 2; a++) count[a] = 1e15; // 무한대 값
int pos1 = vocab_size - 1; // 리프 노드 포인터 (Leaf node pointer)
int pos2 = vocab_size; // 내부 노드 포인터 (Internal node pointer)
// 허프만 트리 구축: 빈도가 낮은 두 노드를 병합 (Build Huffman tree: merge two nodes
for (int a = 0; a < vocab_size - 1; a++) {
// 가장 작은 두 노드 찾기 (Find two smallest nodes)
int min1i, min2i;
if (pos1 >= 0) {
if (count[pos1] < count[pos2]) {
min1i = pos1; pos1--;
} else {
min1i = pos2; pos2++;
}
} else {
min1i = pos2; pos2++;
}
```

```
// 두 번째로 작은 노드 찾기 (Find second smallest node)
if (pos1 >= 0) {
if (count[pos1] < count[pos2]) {
min2i = pos1; pos1--;
} else {
min2i = pos2; pos2++;
}
} else {
min2i = pos2; pos2++;
}
// 새 내부 노드 생성 (Create new internal node)
count[vocab_size + a] = count[min1i] + count[min2i];
parent_node[min1i] = vocab_size + a;
parent_node[min2i] = vocab_size + a;
binary[min2i] = 1; // 오른쪽 자식은 1 (Right child is 1)
}
// 각 단어에 대한 허프만 코드 생성 (Generate Huffman code for each word)
for (int a = 0; a < vocab_size; a++) {
int b = a, i = 0;
char  code[MAX_CODE_LENGTH];
int  point[MAX_CODE_LENGTH];
// 루트까지 경로 추적 (Trace path to root)
while (1) {
```

```
code[i] = binary[b];

point[i] = b;

i++;

b = parent_node[b];

if (b == vocab_size * 2 - 2) break; // 루트 도달 (Reach root)

}

vocab[a].codelen = i;

vocab[a].point[0] = vocab_size - 2;

// 경로를 역순으로 저장 (Store path in reverse order)

for (int b = 0; b < i; b++) {

vocab[a].code[i - b - 1] = code[b];

vocab[a].point[i - b] = point[b] - vocab_size;

}

}

}
```

한국어 비유: 마치 토너먼트 대진표를 만드는 것과 같습니다. 자주 사용되는 단어일수록 결승전에 가깝게 배치하여, 빠르게 찾을 수 있도록 합니다.

4.2 계층적 소프트맥스 학습 (Hierarchical Softmax Training)

```
// 계층적 소프트맥스를 통한 확률 계산 (Probability calculation through hierarchical

if (hs) {

for (int d = 0; d < vocab[word].codelen; d++) {
```

```
real f = 0;

int l2 = vocab[word].point[d] * layer1_size;

// 내적 계산: 은닉층 벡터와 내부 노드 벡터 (Dot product: hidden layer vector and inte

for (int c = 0; c < layer1_size; c++)

f += neu1[c] * syn1[c + l2];

// 시그모이드 함수 (사전 계산된 테이블 사용) (Sigmoid function using pre-computed tab

if (f <= -MAX_EXP) continue;

else  if (f >= MAX_EXP) continue;

else f = expTable[(int)((f + MAX_EXP) * (EXP_TABLE_SIZE / MAX_EXP / 2))];

// 그래디언트 계산 (허프만 코드 기반) (Gradient calculation based on Huffman code)

real g = (1 - vocab[word].code[d] - f) * alpha;

// 역전파 (Backpropagation)

for (int c = 0; c < layer1_size; c++)

neu1e[c] += g * syn1[c + l2];

// 가중치 업데이트 (Weight update)

for (int c = 0; c < layer1_size; c++)

syn1[c + l2] += g * neu1[c];

}

}
```

계층적 소프트맥스의 장점 (Advantages of Hierarchical Softmax):

- **계산 효율성 (Computational Efficiency)**: O(V) → O(log V) 복잡도 감소
- **메모리 효율성 (Memory Efficiency)**: 전체 소프트맥스 계산 불필요

- **빈도 기반 최적화 (Frequency-based Optimization)**: 자주 사용되는 단어는 짧은 경로

한국어 직관적 설명: 전교생 중에서 1등을 찾기 위해 모든 학생을 비교하는 대신, 토너먼트 방식으로 단계별로 경쟁시켜 빠르게 결과를 얻는 것과 같습니다.

5. 네거티브 샘플링 (Negative Sampling) 구현

5.1 유니그램 테이블 구축 (Unigram Table Construction)

네거티브 샘플링을 위한 확률 분포 테이블을 미리 구축합니다:

```
void  InitUnigramTable() {

double train_words_pow = 0;

double power = 0.75; // 빈도의 3/4 제곱 사용 (Use 3/4 power of frequency)

table = (int *)malloc(table_size * sizeof(int));

// 전체 가중 빈도 계산 (Calculate total weighted frequency)

for (int a = 0; a < vocab_size; a++)

train_words_pow += pow(vocab[a].cn, power);

int i = 0;

double d1 = pow(vocab[i].cn, power) / train_words_pow;

// 테이블 구축: 빈도에 비례하여 인덱스 할당 (Build table: assign indices proportional

for (int a = 0; a < table_size; a++) {

table[a] = i;

if (a / (double)table_size > d1) {

i++;

d1 += pow(vocab[i].cn, power) / train_words_pow;
```

```
}
if (i >= vocab_size) i = vocab_size - 1;
}
}
```

유니그램 테이블의 특징 (Characteristics of Unigram Table):

- **크기 (Size)**: 1억 개 엔트리 (빠른 샘플링을 위해)
- **가중치 (Weight)**: 빈도의 3/4 제곱 (희귀 단어 샘플링 확률 증가)
- **별칭 방법 (Alias Method)**: O(1) 시간 복잡도로 샘플링

한국어 비유: 마치 복권 추첨기와 같습니다. 자주 나오는 단어는 많은 공을, 드문 단어는 적은 공을 넣되, 완전히 비례하지는 않게 조정하여 희귀한 단어도 가끔 뽑힐 수 있게 합니다.

5.2 네거티브 샘플링 학습 (Negative Sampling Training)

```
// 네거티브 샘플링 (Negative sampling)
if (negative > 0) {
for (int d = 0; d < negative + 1; d++) {
int target, label;
if (d == 0) {
// 첫 번째는 정답 단어 (positive sample) (First is the correct word - positive sam
target = word;
label = 1;
} else {
// 나머지는 네거티브 샘플 (Rest are negative samples)
```

```
next_random = next_random * 25214903917ULL + 11;

target = table[(next_random >> 16) % table_size];

// 0번 인덱스(</s>) 방지 (Prevent index 0 (</s>))

if (target == 0) target = next_random % (vocab_size - 1) + 1;

if (target == word) continue; // 정답과 같으면 스킵 (Skip if same as correct answe

label = 0;

}

int l2 = target * layer1_size;

real f = 0;

// 내적 계산 (Dot product calculation)

for (int c = 0; c < layer1_size; c++)

f += neu1[c] * syn1neg[c + l2];

// 그래디언트 계산 (Gradient calculation)

real g;

if (f > MAX_EXP) g = (label - 1) * alpha;

else  if (f < -MAX_EXP) g = (label - 0) * alpha;

else g = (label - expTable[(int)((f + MAX_EXP) * (EXP_TABLE_SIZE / MAX_EXP / 2))

// 가중치 업데이트 (Weight update)

for (int c = 0; c < layer1_size; c++)

neu1e[c] += g * syn1neg[c + l2];

for (int c = 0; c < layer1_size; c++)

syn1neg[c + l2] += g * neu1[c];

}
```

```
}
```

네거티브 샘플링의 장점 (Advantages of Negative Sampling):

- **계산 효율성 (Computational Efficiency)**: 소수의 네거티브 샘플만 사용 (보통 5-10개)
- **품질 (Quality)**: 계층적 소프트맥스와 유사한 성능
- **구현 단순성 (Implementation Simplicity)**: 복잡한 트리 구조 불필요

한국어 직관적 설명: 정답을 맞히는 것뿐만 아니라 "이것은 정답이 아니다"라는 것도 함께 학습합니다. 마치 객관식 문제에서 정답 하나와 오답 몇 개를 함께 보여주며 구별하는 법을 가르치는 것과 같습니다.

6. 벡터 연산 구현 분석 (Vector Operation Implementation Analysis)

6.1 Python에서의 벡터 연산 (Vector Operations in Python)

학습된 벡터들은 Python의 `WordVectors` 클래스를 통해 다양한 연산을 수행합니다:

```
class WordVectors(object):

def  __init__(self, vocab, vectors, clusters=None):

self.vocab = vocab # 단어 배열 (Word array)

self.vectors = vectors # 벡터 행렬 (Vector matrix: vocab_size × vector_size)

self.clusters = clusters # 선택적 클러스터 정보 (Optional cluster information)

# 빠른 인덱싱을 위한 해시 테이블 (Hash table for fast indexing)

self.vocab_hash = {}

for i, word in  enumerate(vocab):

self.vocab_hash[word] = i
```

```
def get_vector(self, word):
"""단어의 벡터 반환 (Return vector of word)"""
return self.vectors[self.ix(word)]
def ix(self, word):
"""단어의 인덱스 반환 (Return index of word)"""
return self.vocab_hash[word]
```

6.2 유사도 계산 구현 (Similarity Calculation Implementation)

```
def distance(self, *args, **kwargs):
"""단어들 간의 거리 계산 (Calculate distances between words)"""
metric = kwargs.get("metric", "cosine")
combinations = list(itertools.combinations(args, r=2))
ret = []
for word1, word2 in combinations:
dist = distance(self[word1], self[word2], metric=metric)
ret.append((word1, word2, dist))
return ret

def similar(self, word, n=10, metric="cosine"):
"""유사한 단어 찾기 (Find similar words)"""
return self.closest(self[word], n=n, metric=metric)
```

```
def  closest(self, vector, n=10, metric="cosine"):
"""벡터와 가장 유사한 n개 단어 반환 (Return n words most similar to vector)"""
distances = distance(self.vectors, vector, metric=metric)
best = np.argsort(distances)[::-1][1:n+1] # 자기 자신 제외 (Exclude self)
best_metrics = distances[best]
return best, best_metrics
```

유사도 계산의 핵심 (Core of Similarity Calculation):

- **코사인 유사도 (Cosine Similarity)**: 정규화된 벡터의 내적
- **NumPy 브로드캐스팅 (NumPy Broadcasting)**: 효율적인 벡터 연산
- **정렬 최적화 (Sorting Optimization)**: `argsort` 를 통한 상위 N개 선택

한국어 비유: 마치 성격 테스트 결과를 비교하는 것과 같습니다. 각 사람의 성격을 여러 차원으로 표현하고, 두 사람의 성격이 얼마나 비슷한지 계산합니다.

6.3 아날로지 연산 구현 (Analogy Operation Implementation)

```
def  analogy(self, pos, neg, n=10, metric="cosine"):
"""
아날로지 연산: king - man + woman = queen
pos=['king', 'woman'], neg=['man']
"""
exclude = pos + neg # 입력 단어들은 결과에서 제외 (Exclude input words from results
# 가중 평균 계산 (Calculate weighted average)
pos_vectors = [(word, 1.0) for word in pos]
neg_vectors = [(word, -1.0) for word in neg]
```

```
mean = []

for word, direction in pos_vectors + neg_vectors:

mean.append(direction * self[word])

mean = np.array(mean).mean(axis=0)

# 유사도 계산 및 정렬 (Calculate similarity and sort)

metrics = distance(self.vectors, mean, metric=metric)

best = metrics.argsort()[::-1][:n + len(exclude)]

# 입력 단어들 제외 (Exclude input words)

exclude_idx = [np.where(best == self.ix(word))

for word in exclude if  self.ix(word) in best]

new_best = np.delete(best, exclude_idx)

best_metrics = metrics[new_best]

return new_best[:n], best_metrics[:n]
```

아날로지 연산의 수학적 원리 (Mathematical Principle of Analogy Operation):

```
결과_벡터 = Σ(양수_단어_벡터) - Σ(음수_단어_벡터)

유사도 = cosine_similarity(결과_벡터, 모든_단어_벡터)
```

한국어 직관적 설명: "왕에서 남성성을 빼고 여성성을 더하면 여왕이 된다"는 논리를 벡터 계산으로 구현한 것입니다.

6.4 거리 함수 구현 (Distance Function Implementation)

```
def  distance(a, b, metric="cosine"):
```

```
"""벡터 간 거리 계산 (Calculate distance between vectors)"""
if metric == "cosine":
# word2vec에서는 모든 벡터가 정규화되어 있어 내적 = 코사인 유사도
# (In word2vec, all vectors are normalized, so dot product = cosine similarity)
return np.dot(a, b.T)
raise Exception(f"Unknown metric '{metric}'")


def unitvec(vec):
"""벡터 정규화 (Vector normalization)"""
return (1.0 / np.linalg.norm(vec, ord=2)) * vec
```

정규화의 중요성 (Importance of Normalization):

- **코사인 유사도 (Cosine Similarity)**: 벡터의 방향만 고려, 크기 무시
- **계산 효율성 (Computational Efficiency)**: 정규화된 벡터의 내적 = 코사인 유사도
- **수치 안정성 (Numerical Stability)**: 오버플로우 방지

한국어 비유: 키가 다른 사람들의 그림자 방향을 비교할 때, 그림자의 길이는 무시하고 방향만 비교하는 것과 같습니다.

7. 성능 최적화 기법 (Performance Optimization Techniques)

7.1 메모리 맵핑 (Memory Mapping)

대용량 벡터 파일의 효율적 처리를 위해 메모리 맵핑을 사용합니다:

```
def to_mmap(self, fname):
```

```
"""메모리 맵 파일로 저장 (Save as memory map file)"""
joblib.dump(self, fname)


@classmethod
def from_mmap(cls, fname):
"""메모리 맵 파일에서 로딩 (Load from memory map file)"""
memmaped = joblib.load(fname, mmap_mode="r+")
return cls(vocab=memmaped.vocab, vectors=memmaped.vectors)
```

메모리 맵핑의 장점 (Advantages of Memory Mapping):

- **메모리 효율성 (Memory Efficiency)**: 필요한 부분만 메모리에 로드
- **빠른 접근 (Fast Access)**: 디스크 I/O 최소화
- **공유 메모리 (Shared Memory)**: 여러 프로세스 간 데이터 공유

한국어 비유: 큰 백과사전을 통째로 들고 다니지 않고, 필요한 페이지만 펼쳐보는 것과 같습니다.

7.2 바이너리 저장 형식 (Binary Storage Format)

바이너리 형식의 장점 (Advantages of Binary Format):

- **공간 효율성 (Space Efficiency)**: 텍스트 대비 50% 이상 용량 절약
- **로딩 속도 (Loading Speed)**: 파싱 과정 생략으로 빠른 로딩
- **정확성 (Accuracy)**: 부동소수점 정밀도 보존

7.3 C 코드 최적화 기법 (C Code Optimization Techniques)

7.3.1 사전 계산된 지수 테이블 (Pre-computed Exponential Table)

```
// 시그모이드 함수 사전 계산 (Pre-compute sigmoid function)

real *expTable = (real *)malloc((EXP_TABLE_SIZE + 1) * sizeof(real));

for (int i = 0; i < EXP_TABLE_SIZE; i++) {

expTable[i] = exp((i / (real)EXP_TABLE_SIZE * 2 - 1) * MAX_EXP);

expTable[i] = expTable[i] / (expTable[i] + 1); // sigmoid(x) = exp(x)/(1+exp(x))

}



// 사용 시 (When using)

real f = expTable[(int)((f + MAX_EXP) * (EXP_TABLE_SIZE / MAX_EXP / 2))];
```

한국어 설명: 복잡한 계산을 미리 해두고 표로 만들어 놓으면, 나중에 빠르게 찾아볼 수 있습니다. 마치 구구단표를 외워두는 것과 같습니다.

7.3.2 메모리 정렬 최적화 (Memory Alignment Optimization)

```
// 128바이트 경계 정렬로 SIMD 최적화 (128-byte boundary alignment for SIMD optimiza

posix_memalign((void **)&syn0, 128, vocab_size * layer1_size * sizeof(real));
```

7.3.3 서브샘플링 (Subsampling)

```
// 빈번한 단어의 확률적 제거 (Probabilistic removal of frequent words)

if (sample > 0) {

real ran = (sqrt(vocab[word].cn / (sample * train_words)) + 1) *

(sample * train_words) / vocab[word].cn;
```

```
next_random = next_random * 25214903917ULL + 11;

if (ran < (next_random & 0xFFFF) / 65536.0) continue;

}
```

서브샘플링의 효과 (Effects of Subsampling):

- **학습 속도 향상 (Improved Training Speed)**: 빈번한 단어 처리 감소
- **품질 개선 (Quality Improvement)**: 희귀 단어 학습 기회 증가
- **균형 조정 (Balance Adjustment)**: 빈도 분포 평활화

한국어 비유: "은, 는, 이, 가" 같은 조사는 너무 자주 나와서 별로 중요하지 않으므로, 가끔씩 건너뛰면서 학습합니다.

8. 구현 분석 결론 (Implementation Analysis Conclusion)

8.1 아키텍처 설계의 장점 (Advantages of Architecture Design)

1. **성능 최적화 (Performance Optimization)**: C 코어로 계산 집약적 부분 처리
2. **사용성 (Usability)**: Python 래퍼로 편리한 인터페이스 제공
3. **확장성 (Scalability)**: 멀티스레딩과 메모리 최적화로 대용량 처리
4. **유연성 (Flexibility)**: 다양한 파라미터와 알고리즘 옵션

8.2 핵심 알고리즘의 특징 (Characteristics of Core Algorithms)

1. **Skip-gram vs CBOW**: 서로 다른 학습 목표와 특성
2. **계층적 소프트맥스 (Hierarchical Softmax)**: 허프만 트리로 계산 복잡도 감소
3. **네거티브 샘플링 (Negative Sampling)**: 간단하면서도 효과적인 근사 방법
4. **벡터 연산 (Vector Operations)**: 정규화와 코사인 유사도 기반 의미 계산

8.3 최적화 기법의 효과 (Effects of Optimization Techniques)

1. **메모리 관리 (Memory Management)**: 정렬, 맵핑, 바이너리 형식으로 효율성 극대화
2. **계산 최적화 (Computational Optimization)**: 사전 계산, SIMD, 멀티스레딩 활용
3. **알고리즘 개선 (Algorithm Improvement)**: 서브샘플링, 동적 학습률로 품질 향상

8.4 한국어 사용자를 위한 핵심 포인트 (Key Points for Korean Users)

1. **이해하기 쉬운 설명 (Easy-to-understand Explanations)**: 복잡한 알고리즘을 일상적인 비유로 설명
2. **실용적인 활용 (Practical Applications)**: 한국어 특성을 고려한 구체적인 사용 방법 제시
3. **기술적 정확성 (Technical Accuracy)**: 영문 원어와 한글 설명을 병기하여 정확한 이해 도모

이러한 구현 분석을 통해 word2vec이 단순한 아이디어를 어떻게 효율적이고 실용적인 시스템으로 구현했는지 이해할 수 있습니다. C와 Python의 장점을 결합한 하이브리드 아키텍처는 현대 머신러닝 라이브러리 설계의 좋은 사례가 됩니다.